People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
**Higher School of Computer Science**
**8 May 1945 – Sidi Bel Abbes**




# MASTER'S THESIS

To obtain the diploma of **Master**
Field: **Computer Science**
Specialty: **Information and Web Systems (SIW)**

**Theme**

# Towards Trustworthy SBOM Recovery from Compiled Android Applications

*An Evidence-Based Component Attribution Framework*

Presented by: **[PLACEHOLDER: Student Name]**

Supervisor: **[PLACEHOLDER: Supervisor Name]**

Defense date: **[PLACEHOLDER: DD/MM/YYYY]**

*Academic Year: 2025–2026*

# Dedication

*[PLACEHOLDER: Personal dedication to be written by the student.]*

# Acknowledgements

**[PLACEHOLDER: Personal acknowledgements to the supervisor, jury, family, colleagues, and supporting institutions.]**

# Abstract

Software Bills of Materials are increasingly used to support software supply-chain transparency and vulnerability management. However, a syntactically valid SBOM is not automatically trustworthy, particularly when it is reconstructed from a compiled Android application without source code or dependency manifests. Compilation, optimization, obfuscation, artifact-family aliases, reference-database gaps, and version similarity can weaken or contradict the evidence supporting a component claim.

This thesis studies trustworthy SBOM recovery as an evidence-based software component attribution problem. It organizes existing Android third-party library and binary component-recovery research into taxonomies of techniques, evidence sources, transformation threats, and evaluation practices. Building on the resulting research gaps, it proposes a conceptual framework in which each recovered claim records component identity, version or version set, evidence provenance, evidence quality, transformation risk, confidence, ambiguity, validation class, and permitted downstream security interpretation.

The implemented APK-to-SBOM PFE system is used as a bounded case study. Its controlled, clean and partial source-validated, source-known upper-bound, experiment-only, report-only, and boundary evidence demonstrates why library identity, exact version, reference coverage, matcher recall, and vulnerability authority must remain separate. The proposed framework therefore favors auditable partial knowledge and explicit abstention over unsupported certainty, and defines a research agenda for calibrated evidence fusion, provenance-aware SBOMs, realistic benchmarks, and native or dynamically loaded component recovery.

**Keywords** — software bill of materials, software supply chain, Android, component attribution, provenance, evidence fusion, trustworthiness, uncertainty, vulnerability management.

# Résumé

**[PLACEHOLDER : La version française sera rédigée après validation de la version anglaise de l'abstract.]**

**Mots-clés** — nomenclature logicielle, chaîne d'approvisionnement logicielle, Android, attribution de composants, provenance, fusion de preuves, confiance, incertitude.

# ملخص

سيُستكمل هذا الملخص بعد مراجعة النسخة الإنجليزية واعتماد المصطلحات العلمية المتعلقة بمكونات البرمجيات والأدلة والثقة.

**الكلمات المفتاحية** — مكونات البرمجيات، سلسلة توريد البرمجيات، أندرويد، الأدلة، مصدر المعلومات، الثقة، عدم اليقين.

# Contents

**Dedication** i

**Acknowledgements** ii

**Abstract** iii

**Résumé** iv

**ملخص** v

**Contents** vi

**List of Figures** x

**List of Tables** xi

**List of Acronyms** xii

## I General Introduction 1

### 1 General Introduction 2

1.1 Context . . . . . 2
1.2 Problem Statement . . . . . 2
1.3 Motivation . . . . . 2
1.4 Research Questions . . . . . 3
1.5 Objectives and Contributions . . . . . 3
1.6 Scope . . . . . 4
1.7 Thesis Organization . . . . . 5

## II Background and Research Methodology 6

### 2 Background and Research Methodology 7

2.1 Introduction . . . . . 7

2.2 Software Supply Chains and SBOMs . . . 7
2.2.1 Authoritative and Reconstructed SBOMs . . . 7
2.3 Trust, Evidence, Provenance, and Uncertainty . . . 8
2.3.1 Evidence and Claims . . . 8
2.3.2 Provenance and Independence . . . 8
2.3.3 Uncertainty and Abstention . . . 8
2.4 Android Compiled-Artifact Context . . . 8
2.5 Software Component Attribution . . . 9
2.5.1 Identity, Version, and Security Authority . . . 9
2.5.2 Reference Dependence . . . 9
2.6 Evidence Sources in APKs . . . 9
2.7 Structured Literature Review Methodology . . . 10
2.7.1 Review Questions and Search Strategy . . . 10
2.7.2 Eligibility and Screening . . . 10
2.7.3 Evidence Tiers . . . 10
2.8 Analysis Dimensions . . . 11
2.9 Threats to the Review . . . 11
2.9.1 Search and Selection Bias . . . 11
2.9.2 Availability and Recency . . . 11
2.9.3 Heterogeneous Evaluation . . . 11
2.9.4 Reviewer and Extraction Bias . . . 12
2.10 Conclusion . . . 12

**III State of the Art and Taxonomy 13**

**3 State of the Art and Taxonomy 14**
3.1 Introduction . . . 14
3.2 Taxonomy of Component-Recovery Techniques . . . 14
3.3 Taxonomy of Evidence Sources . . . 16
3.4 Taxonomy of Transformation Threats . . . 17
3.5 Representative Android TPL Approaches . . . 19
3.5.1 From Profile Matching to Structural Boundaries . . . 19
3.5.2 Version-Aware and Vulnerability-Oriented Pipelines . . . 19
3.5.3 Learning-Based Presence Detection . . . 20
3.5.4 Empirical Assessments and Cross-Tool Studies . . . 20
3.6 Evaluation Practices and Ground Truth . . . 21
3.7 Comparative Analysis . . . 21
3.8 Limitations of Existing Research . . . 23
3.9 Research Gap . . . 24
3.10 Conclusion . . . 25

**IV Proposed Framework 26**

**4 Proposed Evidence-Based Trust Framework 27**

4.1 Introduction . . . 27
4.2 Framework Design Principles . . . 27
4.3 Recovered Claim Model . . . 29
4.3.1 Claim Definition . . . 29
4.3.2 Claim Granularity . . . 30
4.4 Evidence Hierarchy . . . 31
4.5 Evidence Quality Dimensions . . . 32
4.6 Evidence Fusion, Conflict, and Abstention . . . 33
4.6.1 Fusion Policy . . . 33
4.6.2 Conflict Handling . . . 33
4.6.3 Confidence Caps . . . 34
4.7 Claim Classes . . . 34
4.8 Validation and Evidence Lanes . . . 35
4.9 Vulnerability-Claim Policy . . . 36
4.10 Recovered Claim Lifecycle . . . 37
4.11 PFE System as a Case Study . . . 38
4.12 Framework Evaluation . . . 40
4.13 Limitations and Research Agenda . . . 40
4.14 Conclusion . . . 41

**V General Conclusion 42**

**5 General Conclusion and Future Directions 43**

5.1 General Conclusion . . . 43
5.2 Answers to the Research Questions . . . 44
5.3 Main Contributions . . . 45
5.4 Limitations . . . 46
5.4.1 Review Scope and Evidence Availability . . . 46
5.4.2 Domain and Ecosystem Scope . . . 46
5.4.3 Uncalibrated Confidence Classes . . . 46
5.4.4 Case-Study Dependence . . . 46
5.4.5 Source and Reference Boundaries . . . 47
5.4.6 Native, Dynamic, and Protected Code . . . 47
5.5 Future Research . . . 47
5.5.1 Calibrated and Set-Valued Claims . . . 48
5.5.2 Independent and Human-Centered Evaluation . . . 48
5.5.3 Interoperable Provenance . . . 48

5.5.4 Broader Component Domains . . . . . 48
5.5.5 Evidence Knowledge Graphs . . . . . 49
5.6 Closing Remarks . . . . . 49

**A Literature Review Protocol** **50**
A.1 Research Questions . . . . . 50
A.2 Search Strategy . . . . . 50
A.2.1 Sources and Scope . . . . . 50
A.2.2 Concept Blocks . . . . . 51
A.3 Inclusion and Exclusion Criteria . . . . . 51
A.3.1 Inclusion . . . . . 51
A.3.2 Exclusion . . . . . 52
A.3.3 Screening and Evidence Tiers . . . . . 52
A.4 Data Extraction Schema . . . . . 52
A.5 Quality Assessment . . . . . 53
A.6 Synthesis Rules . . . . . 53
A.7 Selected Studies . . . . . 54
A.8 Audit and Change Control . . . . . 55

**Bibliography** **56**

# List of Figures

4.1 Conceptual flow of the proposed evidence-based trust framework. . . . . 33

# List of Tables


3.1 Technique taxonomy for Android component recovery. . . . . 15
3.2 Evidence-source taxonomy for reconstructed Android component claims. . 16
3.3 Transformation-threat taxonomy for Android component recovery. . . . . 18
3.4 Evaluation-practice taxonomy in the Android TPL literature. . . . . 21
3.5 Tier A studies classified by recovery role and trust implication. . . . . 22

4.1 Core fields of a reconstructed SBOM claim. . . . . 30
4.2 Evidence hierarchy for reconstructed component claims. . . . . 31
4.3 Evidence quality dimensions used to bound claim confidence. . . . . 32
4.4 Claim classes and permitted downstream use. . . . . 35
4.5 Validation and evidence lanes. . . . . 35
4.6 Vulnerability interpretation policy. . . . . 37
4.7 Mapping PFE observations to framework concepts. . . . . 38

5.1 Answers to the supporting research questions. . . . . 44
5.2 Priority directions for extending the trust framework. . . . . 47

A.1 Search concept blocks . . . . . 51
A.2 Structured extraction fields . . . . . 52
A.3 Frozen seed corpus and current evidence status . . . . . 54

# List of Acronyms

| Acronym | Definition |
|---|---|
| APK | Android Package Kit |
| API | Application Programming Interface |
| CVE | Common Vulnerabilities and Exposures |
| DEX | Dalvik Executable |
| ML | Machine Learning |
| NVD | National Vulnerability Database |
| OSV | Open Source Vulnerabilities |
| PURL | Package URL |
| R8 | Android code shrinker and optimizer |
| SBOM | Software Bill of Materials |
| SCA | Software Composition Analysis |
| SPDX | Software Package Data Exchange |
| TPL | Third-Party Library |

# Part I

# General Introduction

# Chapter 1

# General Introduction

## 1.1 Context

Software supply chains depend on large ecosystems of reusable components. SBOMs improve transparency by recording these components, but many audit scenarios provide only compiled software artifacts. In such cases, an SBOM is reconstructed from indirect and transformed evidence rather than generated from authoritative dependency metadata.

## 1.2 Problem Statement

A recovered component name or version is not trustworthy merely because a tool emitted it. Trust depends on the origin, independence, transformation resilience, version specificity, and validation status of its evidence. Existing Android detection research provides strong technical signals, but lacks a unified model connecting those signals to claim provenance, uncertainty, and permitted vulnerability interpretation.

## 1.3 Motivation

Software transparency is most useful when an inventory can be inspected, challenged, and updated. Build-generated dependency metadata can often provide direct package coordinates and resolved versions, but auditors, incident responders, application stores, and downstream consumers may receive only a compiled artifact. In that setting, reconstructing an SBOM is necessary, yet the result is an inference from residual binary evidence rather than a direct export of the original build graph.

This distinction matters for vulnerability management. A package name may be sufficient to guide an investigation while remaining insufficient for an exact-version or affected-range decision. Conversely, an exact-looking version can be unsafe when the true release is absent from the reference base, when adjacent releases are feature-equivalent, or when optimization has removed the few discriminating features. If these conditions are hidden, a syntactically valid SBOM can encourage unsupported remediation priorities or false assurance.

The research reviewed in this thesis provides many useful signals for component recovery, including names, class and method structure, opcodes, literals, dependency relations,

graphs, metadata, and learned representations. However, technical evidence and evaluation results answer different questions. Library identity, exact-version attribution, reference coverage, matcher recall, source validation, and vulnerability authority must not be collapsed into one confidence value or success rate.

The academic motivation is therefore to formalize how recovered evidence becomes an auditable claim. In this thesis, a reconstructed claim is *trustworthy* when its identity and version strength, supporting and opposing evidence, provenance, transformation exposure, unresolved uncertainty, validation context, and permitted downstream use are visible and mutually consistent. This definition allows partial knowledge and abstention to remain useful outcomes rather than forcing every analysis into an unsupported exact answer.

## 1.4 Research Questions

The central research question is:

> *How can heterogeneous evidence extracted from compiled and obfuscated Android applications be combined to produce trustworthy SBOM component and version claims with explicit provenance, confidence, uncertainty, and validation boundaries?*

This question is developed through six supporting questions:

1. Which evidence sources are available for recovering software components from compiled Android applications?
2. How have existing Android library-detection and binary component-recovery approaches combined these sources?
3. Which factors determine evidence strength for identity and exact-version attribution?
4. How should conflicting, missing, transformed, or source-dependent evidence be represented?
5. What information must accompany a reconstructed SBOM claim for it to be auditable and useful in vulnerability management?
6. How can such a framework be evaluated without overstating accuracy?

## 1.5 Objectives and Contributions

The objective is to connect Android binary component-recovery techniques to a clear model of claim trust. The thesis reviews and classifies prior work, identifies the conditions that strengthen or weaken identity and version evidence, formalizes an evidence-based attribution framework, illustrates the framework through selected PFE observations, and derives directions for further research.

The resulting contributions are:

1. a recovery-oriented taxonomy of techniques, evidence sources, transformation threats, and evaluation practices;

2. operational trust requirements based on provenance, evidence visibility, identity/version separation, uncertainty, validation context, vulnerability authority, and explicit denominators;
3. a structured recovered-claim model covering identity, version, relationships, boundary observations, and downstream interpretation;
4. an evidence hierarchy and fusion policy with conflict handling, confidence caps, bounded ambiguity, and abstention;
5. a validation-lane and vulnerability-authority model that prevents a security interpretation from exceeding its component evidence; and
6. a PFE-informed framework case study and research agenda for broader calibration, validation, interoperability, and component domains.

These are research and framework contributions. The associated APK-to-SBOM software implementation and its detailed engineering evaluation belong to the PFE; this thesis uses selected project evidence only to motivate and examine the proposed concepts.

## 1.6 Scope

The primary domain is static reconstruction of software-component claims from compiled Android APKs. The analysis focuses on DEX-observable JVM/Maven components, versioned reference artifacts, and the implications of recovered identity and version claims for SBOM production and vulnerability management. Android is used as a demanding case because compilation, shrinking, optimization, obfuscation, multidex packaging, artifact variants, and incomplete references can all change the evidence available after build time.

The thesis covers:

- structured review and taxonomy construction for Android third-party library and binary component-recovery research;
- evidence quality, provenance, transformation exposure, ambiguity, and validation boundaries for recovered claims;
- claim classes for exact, bounded, identity-only, report-only, abstained, and boundary outcomes;
- constraints on component-to-vulnerability interpretation; and
- selected PFE observations used as case-study evidence.

The thesis does not propose a second APK detector, repeat the complete PFE implementation description, or present the PFE evaluation as independent master experiments. It does not claim universal probabilistic calibration, complete dependency recovery, or general native package/version attribution. Native shared objects, runtime-loaded code, encrypted payloads, commercial protectors, and other unsupported component domains are represented as boundary categories and future-research problems. The framework is therefore conceptual and evidence-informed: it defines how claim authority should be bounded, but it is not a formal proof that every relevant component can be observed or identified.

## 1.7 Thesis Organization

The remainder of the thesis is organized as follows.

Chapter 2 establishes the conceptual and methodological foundation. It distinguishes authoritative and reconstructed SBOMs, defines evidence, provenance, uncertainty, and software-component attribution, describes the Android compiled-artifact context, and records the structured literature-review method. Appendix A contains the detailed review protocol, selection rules, extraction schema, quality assessment, and audit controls.

Chapter 3 applies that method to the selected literature. It constructs taxonomies of recovery techniques, evidence sources, transformation threats, and evaluation practices, compares representative approaches without treating incompatible reported scores as one leaderboard, and derives the research gap addressed by the thesis.

Chapter 4 presents the principal conceptual contribution: an evidence-based trust framework for reconstructed SBOM claims. It defines the recovered-claim schema, evidence hierarchy, quality dimensions, fusion and conflict policy, claim classes, validation lanes, vulnerability-authority rules, claim lifecycle, and bounded PFE case study.

Chapter 5 answers the research questions, summarizes the contributions and limitations, and develops future directions including empirical calibration, independent validation, provenance-aware exchange, native and dynamic attribution, human-centered evaluation, and evidence knowledge graphs.

# Part II

# Background and Research Methodology

# Chapter 2

# Background and Research Methodology

## 2.1 Introduction

This chapter establishes the concepts and review method used to reason about trustworthy recovery of software components from compiled Android applications. The thesis treats a recovered SBOM as a set of evidence-backed claims rather than as a simple list of names and versions. This distinction requires a common vocabulary for component identity, provenance, uncertainty, and transformation risk, followed by a transparent method for selecting and comparing prior research.

## 2.2 Software Supply Chains and SBOMs

Modern applications assemble code, resources, native binaries, generated artifacts, and transitive dependencies from many independently maintained projects. An SBOM records these components so that operators can reason about inventory, licensing, provenance, and known vulnerabilities. Standards define how such information is represented, but a representation format does not establish that the recorded component is actually present or that its exact version is known.

The framework considers both SPDX and CycloneDX, while using Package URL as a standard package-identity model [1, 2, 3]. These standards therefore define the target vocabulary and exchange structures, while the attribution framework developed in this thesis defines the evidential conditions under which individual fields may be populated.

### 2.2.1 Authoritative and Reconstructed SBOMs

An authoritative build-time SBOM can use dependency resolution metadata, lockfiles, build graphs, and package-manager records. By contrast, an artifact-reconstructed SBOM begins after those authoritative records have been removed or made unavailable. It must infer a component from observable residue in an APK, compare that residue with references, and preserve the uncertainty introduced by compilation and incomplete coverage.

The distinction is central to this thesis. A reconstructed component identity may

be credible even when its exact version is unresolved. Similarly, a syntactically valid CycloneDX or SPDX document may contain semantically weak claims. Trustworthiness is consequently a property of each claim and its evidence trail, not of the file format alone.

## 2.3 Trust, Evidence, Provenance, and Uncertainty

### 2.3.1 Evidence and Claims

An evidence item is an observable fact used to support or challenge a component claim. Examples include a package fragment, an opcode fingerprint, a class-dependency relation, a resource marker, a native-library name, or a source-validated expected dependency. A claim is the interpretation built from one or more such items, for example that a library family is present or that a particular version is the best-supported candidate.

### 2.3.2 Provenance and Independence

Provenance records where evidence came from, how it was extracted, and which transformations may have affected it. Two matching fields are not independent when they are derived from the same underlying artifact feature. Confidence must therefore not be increased merely by counting correlated signals as separate confirmations.

### 2.3.3 Uncertainty and Abstention

Uncertainty can arise from missing evidence, conflicting evidence, incomplete reference coverage, adjacent versions with near-identical code, partial library inclusion, or destructive optimization. The framework allows ambiguity sets and abstention because an explicit unresolved result is more auditable than a forced exact version. This policy also limits downstream vulnerability authority: a vulnerability finding cannot be stronger than the identity and version evidence on which it depends.

## 2.4 Android Compiled-Artifact Context

Android compilation and R8 transformations directly affect the stability and interpretability of recovered evidence [4].

Java or Kotlin source is compiled and transformed into DEX bytecode, while resources and manifests are packaged into an APK. Dependency boundaries that were explicit during the build may disappear when classes are merged into one or more DEX files. R8 can rename identifiers, remove unused code, rewrite control flow, merge classes, inline methods, and change call sites. Packaging may also introduce multidex layouts, shaded or relocated code, native libraries, generated classes, and dynamically loaded components.

These transformations do not affect all evidence equally. Names and package trees are inexpensive but fragile. Opcode, control-flow, call, and dependency structures can survive renaming, yet they are still altered by shrinking and optimization. Resources and metadata can provide independent support, but they may identify only a family rather

than an exact version. The state-of-the-art taxonomy in chapter 3 therefore classifies both evidence sources and the transformations that threaten them.

## 2.5 Software Component Attribution

### 2.5.1 Identity, Version, and Security Authority

Component attribution contains at least three distinct decisions:

1. **identity attribution**: whether a component family or coordinate is present;
2. **version attribution**: which exact version or bounded version set is supported; and
3. **security interpretation**: which vulnerability statements are permitted by the preceding evidence.

A method that performs well at library-family detection does not automatically provide exact-version authority. Likewise, a vulnerability database match is not confirmation that the affected version is present. Keeping these decisions separate prevents a plausible identity match from being silently promoted into an exact and security-critical statement.

### 2.5.2 Reference Dependence

Many attribution methods compare an application with profiles extracted from known library releases. Such a method can recognize only what its reference corpus represents. Failure may therefore result from the matcher, the extractor, or a missing reference rather than from absence of the component. Reference coverage is treated as a first-class validity condition throughout the thesis.

## 2.6 Evidence Sources in APKs

The review extracts evidence into recurring families:

- lexical and namespace evidence, including package and class names;
- bytecode and semantic evidence, including opcodes, API references, literals, and method descriptors;
- structural evidence, including control-flow, call, inheritance, and class-dependency graphs;
- resource and packaging evidence, including manifests, resource names, assets, and native-library files;
- external reference and metadata evidence, including library profiles, Maven coordinates, dependency relations, and release chronology;
- learned representations derived from bytecode or structural features;

- validation evidence, including controlled builds, source manifests, and source-constrained observations.

No family is assumed to be universally authoritative. The framework evaluates its origin, transformation resilience, version specificity, independence, coverage, and validation status.

## 2.7 Structured Literature Review Methodology

The literature study follows a frozen structured mapping protocol adapted from software-engineering systematic-review guidance [5]. It is not presented as a completed exhaustive systematic literature review because the initial corpus was assembled iteratively before the protocol freeze. Instead, the protocol makes the existing corpus auditable and governs all future search refreshes, screening, extraction, and synthesis.

### 2.7.1 Review Questions and Search Strategy

The review asks which evidence sources and attribution techniques are used, which Android transformations threaten them, how identity and version results are evaluated, how uncertainty and provenance are represented, and which gaps prevent trustworthy SBOM claims. Searches combine four concept blocks: Android artifacts; third-party components or SBOMs; detection or attribution tasks; and obfuscation, optimization, version, or vulnerability terms.

Primary sources include ACM Digital Library, IEEE Xplore, SpringerLink, ScienceDirect, USENIX, and arXiv. Google Scholar is restricted to discovery and citation chaining. The date scope begins in 2015 and continues through the recorded search date. Full query text and database adaptations are preserved in chapter A.

### 2.7.2 Eligibility and Screening

Eligible studies propose or evaluate Android binary component attribution, version identification, relevant benchmarks, empirical assessments, or SBOM-oriented recovery frameworks. Studies based only on authoritative source manifests, unrelated malware classification, product pages, duplicates, and records without extractable methodology are excluded.

Screening proceeds through deduplication, title-and-abstract screening, full-text assessment, and evidence-tier assignment. Related conference and journal publications are treated as one study family unless the later publication contributes distinct methods, datasets, or results.

### 2.7.3 Evidence Tiers

The frozen seed corpus contains 20 publication units. Thirteen have locally available full texts and form *Tier A*. Seven are *Tier B* supplementary records whose metadata or structured notes are available but whose local full texts have not yet been verified. Tier B records may inform search coverage and provisional gap mapping, but they cannot

independently support precise quantitative conclusions. Standards and official platform documentation form contextual *Tier C* and are not counted as primary studies.

## 2.8 Analysis Dimensions

Each included study is extracted across common dimensions: task granularity, input assumptions, segmentation, evidence families, matching or learning method, transformation coverage, ground truth, dataset composition, metrics, confidence and ambiguity behavior, reference dependence, artifact availability, SBOM output, vulnerability mapping, and stated limitations.

Tier A studies are additionally assessed against eight reporting-quality questions covering objective clarity, reproducibility, ground truth, metric definition, transformation conditions, identity/version separation, artifact availability, and threats to validity. Quality affects the weight of a study in synthesis but does not automatically erase historically important work.

Synthesis is narrative and taxonomy-based. Metrics are compared only when their task unit, dataset, ground truth, and transformation conditions are compatible. Original-paper results remain separate from later independent benchmarks, and library-level, class-level, and version-level results are never merged into a single ranking.

## 2.9 Threats to the Review

### 2.9.1 Search and Selection Bias

The pre-protocol corpus was built iteratively and may overrepresent well-known Android security venues or studies connected through citation chains. The frozen search blocks, multiple publisher databases, and explicit refresh log reduce but do not eliminate this risk.

### 2.9.2 Availability and Recency

Several recent or emerging studies currently exist only as supplementary records. Treating them as Tier B prevents incomplete metadata from acquiring the same authority as inspected full text. Their classifications may change after verification, and all such changes must be logged.

### 2.9.3 Heterogeneous Evaluation

Reported accuracy values often refer to different task definitions, datasets, library versions, obfuscators, and ground-truth procedures. Direct numerical ranking can therefore be misleading. The review preserves these conditions and prefers qualified comparison over pooled scores.

### 2.9.4 Reviewer and Extraction Bias

The current review is primarily maintained by one researcher. A fixed extraction schema, explicit `NR` values for unreported information, source citations, and retained structured records provide an audit trail. A later independent screening sample would further strengthen reliability.

## 2.10 Conclusion

This chapter separated a valid SBOM representation from the evidential strength of its contents and defined component attribution as a sequence of identity, version, and security-authority decisions. It also froze a structured mapping protocol that prevents unavailable full texts, incompatible metrics, and correlated signals from being silently promoted into strong claims. The next chapter applies these rules to build the state-of-the-art taxonomies that motivate the proposed trust framework.

# Part III

# State of the Art and Taxonomy

# Chapter 3

# State of the Art and Taxonomy

## 3.1 Introduction

This chapter applies the structured mapping protocol defined in chapter 2 and chapter A to the Android third-party library detection literature. Its purpose is not to produce a numerical leaderboard. The reviewed studies use different datasets, task units, obfuscation settings, and ground-truth procedures, so a direct ranking would hide the very uncertainty that this thesis aims to expose. Instead, the chapter organizes prior work by technique, evidence source, transformation threat, and evaluation practice.

The analysis is centered on the 13 Tier A studies whose full texts are available locally. Tier B records are useful for search coverage and for identifying emerging directions, especially SBOM-oriented recovery and recent benchmarking, but they are not used here as independent authority for precise quantitative claims. This boundary keeps the review consistent with the protocol and prevents unverified metadata from receiving the same weight as inspected studies. Fields not reported by a Tier A publication are treated as not reported rather than inferred, and original-paper claims are kept separate from later cross-tool observations.

The chapter uses the term *component recovery* broadly. A study may recover a library family, a package cluster, a class set, an exact version, a vulnerability-related version list, or only a presence signal. For trustworthy SBOM reconstruction, these outputs are not equivalent. A method that detects the presence of a library family does not necessarily justify an exact Maven version, and an exact-version candidate does not automatically justify a CVE claim unless the evidence, reference coverage, and validation context are visible.

## 3.2 Taxonomy of Component-Recovery Techniques

The reviewed approaches can be grouped into seven recurring technique families. The categories are not mutually exclusive: most mature tools combine several families, for example by using a fast coarse filter followed by a slower structural match. The taxonomy therefore describes the role each family plays in a recovered component claim.

**Table 3.1:** Technique taxonomy for Android component recovery.

| Technique family | Typical evidence | Trust role | Tier A examples |
|---|---|---|---|
| Namespace and metadata heuristics | Package trees, class names, library naming conventions | Cheap candidate generation, but weak authority when names are renamed, flattened, relocated, or shaded. | Assessed in [6, 7] |
| Profile, hash, and signature matching | Method descriptors, Merkle trees, fuzzy hashes, class or method signatures | Can support identity and, when references are complete, version claims. Its authority is bounded by reference coverage and by the stability of the hashed features under optimization. | LibScout [8], ORLIS [9], LibID [10] |
| Structural graph segmentation | Call, inheritance, containment, and class-dependency relations | Improves module boundary recovery when package structure is unreliable. Supports stronger identity evidence, but does not by itself solve exact version attribution. | LibD [11], LibPecker [12], assessment studies [6, 7] |
| Coarse-to-fine matching | Fast filtering followed by detailed opcode, CFG, or structural comparison | Balances scalability and precision. Trust depends on both stages: a missed coarse candidate can cap recall, while an over-permissive fine stage can inflate false positives. | ORLIS [9], ATVHUNTER [13], LibScan [14] |
| Version-discriminating semantic features | Opcode sequences, basic-block features, call chains, field-operation patterns, literals, and class summaries | Provides the main route from library identity to exact-version authority. Adjacent versions and optimized builds remain difficult because many versions share most code. | ATVHUNTER [13], LibScan [14], LibHunter [15], SAD [16] |
| Learned representations | Smali token sequences, structural vocabularies, neural embeddings | Can learn obfuscation-resilient presence signals, but the reviewed Tier A learning approach does not provide exact-version or boundary authority on its own. | LibAttention [17] |

| Technique family | Typical evidence | Trust role | Tier A examples |
|---|---|---|---|
| Empirical assessment and benchmarking | Shared datasets, patched tools, source-known applications, evaluation criteria | Does not directly recover components, but defines which claims are comparable and exposes tool failure modes, missing denominators, and reproducibility limits. | Tool assessments [6, 7], Ramirez thesis [18] |

The movement across these families is historically important. Early methods made library profiles and package structure usable at scale, but later studies increasingly treat package names as unreliable evidence. Graph-based and version-aware methods improve the situation by using relations, opcode content, and class behavior. The newest verified Tier A work adds learned representations and finer class-level summaries. Even so, the taxonomy shows that standard SBOM emission remains outside the main output model of the verified Android TPL literature: most studies output names, clusters, versions, class mappings, or vulnerability-linked lists rather than a provenance-rich CycloneDX or SPDX document.

## 3.3 Taxonomy of Evidence Sources

An evidence source is useful for trustworthy SBOM recovery only when its origin, stability, version specificity, and provenance are understood. The same signal may be strong for identity and weak for exact version. Conversely, a highly version-specific literal may be too fragile to survive resource removal or string encryption. Table 3.2 summarizes the recurring evidence families extracted from the Tier A studies.

**Table 3.2:** Evidence-source taxonomy for reconstructed Android component claims.

| Evidence family | Examples | Typical stability | Trust implication |
|---|---|---|---|
| Names and namespaces | Package prefixes, class names, manifest names | Weak under renaming, package flattening, relocation, and shading | Useful for fast hints and human inspection, but insufficient for exact SBOM claims without corroboration. |
| Descriptors and API surfaces | Method signatures, fuzzy type descriptors, Android API usage | More stable than names under identifier renaming; weaker under dead-code removal and inlining | Supports identity and coarse version discrimination when combined with reference profiles. |

| Evidence family | Examples | Typical stability | Trust implication |
|---|---|---|---|
| Opcode and instruction sequences | Basic-block opcodes, method opcode chains, call-chain opcodes | Resilient to naming changes; threatened by optimization, inlining, and control-flow rewriting | Provides version-discriminating evidence, especially when compared against known library releases. |
| Control-flow and dependency structures | CFGs, call graphs, inheritance graphs, class-dependency graphs | Stable against simple renaming and often against package restructuring; affected by optimization and partial inclusion | Improves module boundary recovery and reduces dependence on package layout. |
| Literals, constants, and resources | Strings, constants, resource identifiers, assets | Potentially version-specific, but fragile under stripping, string encryption, and resource rewriting | Best used as tie-breakers or independent corroboration, not as sole authority. |
| Reference corpus and release metadata | Library profiles, Maven coordinates, release chronology, known vulnerable version ranges | External to the APK; stable only if the corpus is complete and correctly constructed | Defines the ceiling for recognizable identities and versions. Missing references must not be interpreted as component absence. |
| Learned representations | Transformer embeddings over Smali tokens, structural vocabularies | Can absorb complex obfuscation patterns, but may be hard to audit feature by feature | Promising for presence screening and robustness, but provenance and exact version authority require additional explanation. |
| Validation evidence | Controlled builds, source manifests, F-Droid dependency data, source-known ground truth | Strong within the validation context; not automatically generalizable | Can upgrade a claim's evaluation status, but must remain separate from APK-only inference. |

The trust implication of this taxonomy is that evidence fusion should prefer independent and complementary evidence over repeated variants of the same feature. For example, several hash matches derived from the same method body are less independent than a method-level match corroborated by a class-dependency relation and a version-specific literal. The framework in chapter 4 builds on this distinction by treating independence, provenance, and reference coverage as first-class quality dimensions.

## 3.4 Taxonomy of Transformation Threats

Android component recovery is difficult because compilation and packaging can remove the very boundaries that build-time dependency tools rely on. R8 and related obfuscators

may rename identifiers, flatten packages, remove unused code, rewrite control flow, and inline methods [4]. The reviewed literature shows that no single evidence family remains stable under all transformations.

**Table 3.3:** Transformation-threat taxonomy for Android component recovery.

| Threat | Evidence most affected | More resilient evidence | Trust consequence |
|---|---|---|---|
| Identifier renaming | Class, method, field, and package names | Fuzzy descriptors, opcode features, structural relations | Name matches cannot be treated as independent authority unless corroborated. |
| Package flattening and relocation | Package-subtree segmentation and namespace heuristics | Class-dependency graphs, call relations, profile matching independent of package roots | Package-based module boundaries become unreliable; graph segmentation becomes more important. |
| Control-flow rewriting | CFG shape, branch structure, fuzzy CFG hashes | Higher-level dependency relations, method descriptors, class summaries | CFG-only methods may overstate robustness when control-flow transformations are not in the evaluation. |
| Dead-code removal and shrinking | Presence of optional classes, methods, API calls, literals | Partial-match scoring, class-level confidence, source-known validation | Absence of a feature may indicate transformation rather than absence of the library. |
| Method inlining and outlining | Method boundaries, call-chain evidence, version-specific method bodies | Multi-scale evidence, class-level summaries, literals, residual dependency structure | Exact-version authority must be capped when inlining changes the features used for version discrimination. |
| Multidex and split packaging | Single-file DEX assumptions and incomplete scanners | Whole-APK extraction over all DEX files and split artifacts | Evaluation denominators must state whether all executable artifacts were analyzed. |
| Partial library inclusion | Binary present/absent decisions and complete-profile matching | Similarity scores, per-class evidence, explicit ambiguity classes | A partial match should not automatically become a full component/version claim. |
| Nested or transitive dependencies | Flat library lists and one-library-one-package assumptions | Dependency graphs, source-assisted validation, relationship-aware SBOM models | A recovered component may be direct, transitive, bundled, shaded, or copied; the relationship must remain uncertain unless evidenced. |
| Native and dynamic code | DEX-only static features | Native file hashes, load-site evidence, dynamic analysis, source validation | Out-of-scope code must be represented as a boundary, not silently ignored. |

The most consequential threats for exact-version claims are shrinking, partial inclusion, and method inlining. Identifier renaming is comparatively well handled by modern structural or opcode-based approaches, but version evidence often depends on small differences between adjacent releases. Optimizations that delete, merge, or inline exactly those differences can turn an apparently confident match into a version-band ambiguity.

## 3.5 Representative Android TPL Approaches

### 3.5.1 From Profile Matching to Structural Boundaries

LibScout established a reference-profile model for matching compiled Android applications against known library releases. Its three-layer profile structure and fuzzy method descriptors made identifier renaming less damaging, and its similarity scores provided a graded alternative to exact profile equality [8]. From the viewpoint of trustworthy SBOM recovery, the important lesson is dual: reference profiles can support version claims, but those claims inherit the assumptions of the profile structure and the reference corpus. When package layout or method boundaries are transformed, the confidence of the resulting claim must be reduced rather than silently preserved.

LibD moved away from pure package-layout dependence by recovering library instances through a homogeny graph over inclusion, inheritance, and call relations, combined with opcode-derived fingerprints [11]. This made large-scale library discovery more resilient to naming changes, but the output remained a clustering-oriented library identification result rather than an exact-version or SBOM result. For the taxonomy, LibD is therefore strong evidence for graph-based boundary recovery and weak evidence for version authority.

ORLIS introduced a two-stage obfuscation-resilient design: a coarse filter narrows candidates, and a fine CFG-based fuzzy comparison performs more expensive matching [9]. The design is valuable because it separates scalability from detailed confirmation, and its Jaccard-style similarity naturally supports partial evidence. Its limitations are equally important for the proposed framework: a structural match is not automatically a version claim, and computational cost affects whether a method can be used as routine SBOM evidence.

LibPecker and LibID further illustrate the shift from names to structural and class-level evidence. LibPecker emphasizes class dependency and adaptive similarity. LibID formulates consistent class correspondence with binary integer programming and identifies specific library versions under code shrinking and package modification [12, 10]. Together with the later assessment studies, they support the review's central classification: package structure is a fragile feature family, while class relationships, signatures, and opcode-derived evidence are more suitable as identity support under obfuscation.

### 3.5.2 Version-Aware and Vulnerability-Oriented Pipelines

ATVHUNTER combines class-dependency based decoupling, coarse CFG filtering, fine-grained opcode matching, and a vulnerability database to identify vulnerable library versions [13]. It is one of the most important verified studies for this thesis because it connects version identification to security interpretation. However, its output remains a version and vulnerability list rather than a standard SBOM with explicit provenance,

uncertainty, and claim boundaries. The study therefore motivates the framework's rule that vulnerability authority cannot exceed the strength of the underlying version evidence.

LibScan and LibHunter extend the version-aware family by using richer opcode, class-correspondence, call-chain, and optimization-aware evidence [14, 15]. Their relevance is not only that they improve matching. They also show that code optimization itself is an evaluation dimension. In particular, LibHunter augments reference features to model call-site optimization and performs cross-inlining method matching. A method evaluated only on clear or lightly obfuscated applications has not demonstrated the same authority as a method evaluated under inlining and shrinking.

SAD provides the most detailed Tier A example of class-structural version-level reasoning. It splits classes into stateful and stateless categories, applies field-operation sequences or opcode similarity accordingly, and uses literals to break ties between adjacent versions [16]. This design supports the thesis claim that exact-version attribution requires evidence at a finer granularity than library presence. At the same time, its remaining sensitivity to method inlining confirms that strong reported version-level performance does not eliminate transformation risk.

### 3.5.3 Learning-Based Presence Detection

LibAttention represents a different family: instead of comparing explicit profiles, it learns representations from large Smali corpora and predicts library presence with a Transformer-based model [17]. This is important because learned features can capture patterns that rule-based signatures miss under aggressive obfuscation. For SBOM reconstruction, however, the reviewed Tier A result is bounded: it predicts presence rather than exact version, does not expose an explicit class or library-boundary mapping, and does not provide standard SBOM output. In the taxonomy, learned presence detection is therefore a potentially useful candidate-generation signal but not sufficient version or vulnerability authority by itself.

### 3.5.4 Empirical Assessments and Cross-Tool Studies

The ASE 2020 assessment and the later systematic assessment are not merely background surveys; they define several trust requirements for any recovered SBOM pipeline [6, 7]. They show that tool results depend on module decoupling, multidex handling, AAR processing, obfuscation setting, version-identification capability, efficiency, and ease of reproduction. These studies are a warning against reporting a single accuracy value without denominators, exclusions, and task levels.

The Ramirez thesis adds a source-validation perspective by comparing Android and iOS detection tools and using open-source Android applications as a ground-truth basis [18]. For this master thesis, its main contribution is methodological: source-known applications can validate specific outputs, but that validation is a distinct evidence lane and should not be merged with APK-only inference.

## 3.6 Evaluation Practices and Ground Truth

The reviewed studies evaluate different questions even when they use similar words such as detection, identification, precision, recall, or version accuracy. A trustworthy synthesis must therefore classify evaluation practice before interpreting the metric.

**Table 3.4:** Evaluation-practice taxonomy in the Android TPL literature.

| Evaluation practice | What it can show | Boundary for SBOM trust |
|---|---|---|
| Controlled library or reference corpus | Whether a matcher can distinguish known library profiles or versions under specified transformations | Results depend on reference completeness and may overstate performance on diverse real applications. |
| Synthetic or tool-based obfuscation tests | Whether selected transformations damage a feature family | Only covers the transformations and configurations tested; R8 inlining and shrinking require explicit treatment. |
| Market-scale discovery | Whether a detector scales and finds plausible library families in many apps | Without source ground truth, discovery volume is not the same as correctness or exact-version authority. |
| Open-source or source-known validation | Whether recovered outputs match dependency evidence available outside the APK | Strong for audited cases, but it is a validation lane, not proof that the same APK-only method is generally correct. |
| Cross-tool benchmark or assessment | Which tools fail under shared criteria and where methodology is inconsistent | Useful for gap analysis, but metrics remain bounded by the benchmark's dataset and task definitions. |
| Security or vulnerability mapping | Whether detected versions can be connected to vulnerability data | The vulnerability conclusion is valid only as far as the component identity, exact version, and vulnerability database mapping are valid. |

This taxonomy explains why the chapter does not pool reported performance numbers. LibScout's profile precision, ORLIS's obfuscation precision, LibD's market-scale discovery volume, ATVHUNTER's version-level results, LibAttention's R8-resilient presence detection, and SAD's class-structural version scores answer different questions [8, 11, 9, 13, 17, 16]. For a reconstructed SBOM, the evaluation question must be restated as: *which claim type is being validated, against which denominator, and under which transformations?*

## 3.7 Comparative Analysis

Table 3.5 applies the chapter taxonomy to the Tier A seed corpus. The final column states the trust interpretation rather than a simple score. This is deliberate: the same study can be strong evidence for one claim type and weak evidence for another. The table records only supported roles and limitations; missing study fields are not filled by inference.

**Table 3.5:** Tier A studies classified by recovery role and trust implication.

| Study | Primary role | Main evidence and technique | SBOM-trust implication |
|---|---|---|---|
| LibScout [8] | Reference-profile identity and version matching | Fuzzy method descriptors, package-class-method profile hierarchy, similarity score | Supports the idea of version profiles, but package and method-structure assumptions must be exposed as evidence limits. |
| LibD [11] | Scalable library discovery and boundary recovery | Homogeny graph, call/inheritance/inclusion relations, opcode-derived fingerprints | Strengthens identity evidence under name obfuscation, but does not provide exact-version or SBOM authority. |
| LibPecker [12] | Class-dependency based detection | Class dependency hashing, adaptive similarity, weighted class matching | Shows that structural class relations are useful, while similarity still needs explicit thresholds and uncertainty handling. |
| ORLIS [9] | Obfuscation-resilient structural matching | Coarse filtering, CFG fuzzy hashing, Jaccard similarity | Provides partial-match evidence, but high cost and missing version authority limit direct SBOM use. |
| LibID [10] | Obfuscated library and version identification | Consistent class correspondence through binary integer programming, with source-backed threshold evaluation | Supports specific-version claims under its tested transformations; downstream security authority still depends on reference and vulnerability evidence. |
| ASE 2020 assessment [6] | Cross-tool failure analysis | Unified criteria, tool patches, multidex and AAR observations | Defines required evaluation dimensions and warns that engineering blind spots can dominate measured accuracy. |
| Systematic assessment [7] | Broader comparative methodology | Six evaluation criteria, module-decoupling comparison, tool availability and limitations | Supports taxonomy-based synthesis and shows that CDG-style evidence is preferable to package-only evidence. |
| ATVHUNTER [13] | Version and vulnerability-oriented detection | CDG decoupling, CFG coarse filter, opcode fuzzy matching, vulnerability DB | Strongly connects version evidence to security use, but still lacks standard SBOM provenance and full R8-inlining coverage. |

| Study | Primary role | Main evidence and technique | SBOM-trust implication |
|---|---|---|---|
| LibScan [14] | Precise TPL identification | Class correspondence, method opcode similarity, call-chain evidence | Improves fine-grained matching evidence; claim strength remains tied to reference coverage and transformation conditions. |
| Ramirez thesis [18] | Cross-platform and source-known evaluation | Tool comparison, F-Droid ground truth, Android/iOS contrast | Shows the value of source validation as a separate lane and confirms that SBOM output is not solved by existing detectors. |
| LibHunter [15] | Optimization-aware detection | Call-site-optimization feature enhancement and cross-inlining method matching | Makes optimization a first-class validity condition for exact-version claims. |
| LibAttention [17] | Learning-based presence detection | Transformer over Smali tokens and SDK structural vocabulary | Promising for obfuscation-resilient identity hints, but insufficient alone for exact versions, provenance, or CVE authority. |
| SAD [16] | Class-structural version-level detection | CDG neighbor aggregation, stateful/stateless split, field-operation sequence, literal tie-breakers | Provides strong version-discriminating evidence, while remaining sensitive to method inlining and lacking standard SBOM output. |

Three patterns emerge from this comparison. First, the strongest exact-version approaches are multi-evidence approaches: they do not rely on names alone but combine structural, opcode, class, and literal information. Second, evaluation authority is not the same as detection authority. Assessment studies and source-known validation do not identify libraries by themselves, but they determine whether a result can be interpreted safely. Third, standard SBOM trust requirements are largely external to the detector output: provenance, ambiguity, validation class, relationship type, and vulnerability authority are rarely represented as first-class fields.

## 3.8 Limitations of Existing Research

The taxonomy exposes limitations that recur across the verified literature:

1. **Library identity and exact version are often conflated.** Presence detection, class-set matching, and exact-version attribution are different claims, but tool outputs do not always preserve the distinction.

2. **Reference coverage is rarely treated as part of the result.** A missed match can result from absent references, extraction failure, or a weak matcher. Without coverage metadata, the user cannot tell which failure occurred.

3. **Transformation coverage is incomplete.** Identifier renaming is widely considered, but shrinking, partial inclusion, package flattening, and method inlining remain unevenly evaluated [15, 17, 16].

4. **Confidence is usually a threshold outcome, not an auditable explanation.** Many methods compute similarities or classifier scores, but few expose why a claim should be trusted, which signals conflicted, or why the method did not abstain.

5. **Partial libraries and dependency relationships are not fully represented.** Similarity scores can detect partial inclusion, but a standard SBOM also needs relationship semantics: direct, transitive, bundled, shaded, copied, or uncertain.

6. **Vulnerability interpretation is stronger than the underlying evidence in many pipelines.** A detected library name is not enough for a version-specific CVE claim. Even a plausible version candidate requires reference, transformation, and validation context.

7. **Reproducibility remains uneven.** Tool availability, installation difficulty, dataset access, and benchmark construction vary across studies, limiting independent confirmation of reported results.

8. **Standard SBOM output is not the dominant research artifact.** The verified studies mostly produce detector outputs rather than complete CycloneDX or SPDX statements with evidence, provenance, and uncertainty fields.

These limitations do not make the existing work weak. Rather, they show that the literature has mostly optimized detection performance, while trustworthy SBOM recovery requires an additional layer that explains what each recovered claim means and how it may be used.

## 3.9 Research Gap

The state of the art provides valuable techniques for recognizing Android third-party libraries from compiled artifacts. Reference profiles, fuzzy descriptors, dependency graphs, opcode features, call-chain evidence, stateful/stateless class summaries, learned Smali representations, and source-known validation all contribute useful evidence. However, the field still lacks a unified model that connects these signals to a reconstructed SBOM claim.

The research gap can therefore be stated as follows:

> Existing Android component detectors provide useful identification and version-discrimination techniques, but they do not consistently attach each recovered SBOM claim to its evidence provenance, reference coverage, transformation risk, confidence boundary, ambiguity, validation class, and permitted vulnerability interpretation.

This gap is not only a formatting problem. A syntactically valid SBOM can still be untrustworthy if it hides uncertainty, promotes identity evidence into exact-version authority, or turns an ambiguous version set into a definitive CVE finding. Conversely, a detector that abstains or reports a bounded version set may be more useful for security decision-making than a pipeline that always emits a single version without explaining the evidence.

## 3.10 Conclusion

This chapter classified Android component-recovery research by technique, evidence source, transformation threat, and evaluation practice. The synthesis shows a clear technical progression from package and profile matching toward graph-based, version-aware, optimization-aware, and learning-based methods. It also shows a persistent trust gap: detector outputs rarely preserve enough provenance, uncertainty, validation context, and security authority to become auditable reconstructed SBOM claims on their own.

The next chapter responds to this gap by proposing an evidence-based trust framework. The framework does not replace the reviewed techniques. Instead, it defines how their outputs should be represented, fused, bounded, and interpreted so that a recovered component entry communicates not only what was detected, but why it should be believed and how safely it may be used.

# Part IV

# Proposed Framework

# Chapter 4

# Proposed Evidence-Based Trust Framework

## 4.1 Introduction

The previous chapter showed that Android third-party library detection has advanced from package-name heuristics toward structural, graph-based, optimization-aware, and learning-assisted methods. These techniques are valuable, but they do not by themselves answer the trust question required by an SBOM: when a compiled APK yields a component name, a candidate version, or a vulnerability warning, what exactly has been proven, what remains uncertain, and how safely may the result be used?

This chapter proposes an evidence-based trust framework for reconstructed SBOM claims. The framework is not a replacement for existing detection methods such as dependency-graph matching, opcode similarity, learned Smali features, or source-known validation [7, 13, 17, 16]. Instead, it defines a layer above those methods. That layer converts detector outputs into explicit claims with evidence, provenance, confidence boundaries, ambiguity, and permitted downstream interpretation.

The central idea is simple: every recovered component entry should be treated as a claim, not as an unqualified fact. A claim may be exact, bounded, identity-only, report-only, or out-of-scope. It may be supported by direct bytecode evidence, by independent structural evidence, by source validation, or only by contextual hints. It may also be limited by missing references, partial inclusion, R8 optimization, dependency shading, native code, or hidden runtime loading. The framework makes these distinctions visible so that an SBOM user can understand not only what was emitted, but why it was emitted and what it is allowed to mean.

## 4.2 Framework Design Principles

The framework is built around ten design principles. They are intentionally conservative because reconstructed SBOMs are often used for security decisions, compliance checks, and vulnerability triage.

**P1. Claim explicitness.** Every emitted component, version, relationship, or vulnerability finding is represented as a claim with an identifier, a claim

type, supporting evidence, and a current state. The pipeline does not silently transform a similarity score into a fact.

**P2. Identity-version separation.** Component identity and exact version are separate claims. Evidence that supports the presence of `okhttp` or `androidx.core` does not automatically support a specific version. This separation is essential for CVE interpretation because most advisories are version-dependent.

**P3. Evidence provenance.** Each signal records where it came from: APK-side bytecode, manifest metadata, resource evidence, reference database profiles, source-assisted validation, external package metadata, vulnerability databases, or report-only model output.

**P4. Reference coverage awareness.** A negative or weak result is interpreted in relation to the available reference database. Missing coordinates, missing versions, failed feature extraction, and incomplete profile families are treated as coverage risks, not as proof that a component is absent.

**P5. Transformation-bounded confidence.** Confidence is capped when the analyzed artifact is affected by transformations that the evidence cannot reliably handle, including package renaming, code shrinking, partial inclusion, repackaging, multidex splitting, class merging, and method inlining [4, 15].

**P6. Conflict preservation.** Competing versions, adjacent library families, or disagreeing evidence sources remain visible. The framework prefers a bounded ambiguity set or an abstention over a false sense of exactness.

**P7. Validation-lane separation.** Controlled fixtures, clean source-validated real applications, partial source-validated applications, source-known upper bounds, diagnostic experiments, and limitation cases are separate lanes. Scores from one lane must not be used as if they prove another lane.

**P8. Authority monotonicity.** Downstream authority cannot exceed upstream evidence. A vulnerability claim cannot be stronger than the component and version claim on which it depends; a standard CycloneDX or SPDX entry cannot hide unresolved ambiguity behind a syntactically valid package URL [1, 2, 3].

**P9. Report-only containment.** Diagnostic or machine-learning assistance may guide investigation, ranking, and future work, but it is not allowed to modify authoritative component, version, or vulnerability outputs until it passes the same validation and regression gates as the deterministic evidence.

**P10. Auditable abstention.** Not emitting a component, emitting identity without version, or emitting a version band is a legitimate outcome when evidence is insufficient. The framework records why the pipeline abstained so that uncertainty becomes a visible result rather than a silent failure.

## 4.3 Recovered Claim Model

### 4.3.1 Claim Definition

A recovered claim is a structured statement about a component-related fact in an analyzed APK. The claim may concern component identity, exact version, version range, artifact family, dependency relationship, file evidence, or vulnerability exposure. Formally, a claim can be described as:

$$C = \langle s, t, a, e, p, r, q, u, v, \alpha \rangle$$

where $s$ is the analyzed subject, $t$ is the claim type, $a$ is the asserted value, $e$ is the supporting evidence set, $p$ is provenance, $r$ is reference and transformation risk, $q$ is the evidence quality profile, $u$ is uncertainty or ambiguity, $v$ is validation context, and $\alpha$ is the permitted downstream authority. This notation is not meant to impose a specific storage format. It is a discipline for preventing weak evidence from being consumed as a stronger result.

**Table 4.1:** Core fields of a reconstructed SBOM claim.

| Field | Question answered | Examples |
|---|---|---|
| Subject | What artifact is being analyzed? | APK identifier, package name, hash, build variant, DEX or native file scope. |
| Claim type | What kind of statement is this? | Component identity, exact version, bounded version set, relationship, file component, vulnerability status. |
| Asserted value | What is being claimed? | Maven coordinate, PURL, version, candidate family, dependency edge, native file hash. |
| Evidence set | Why is the claim supported? | Class dependency graph, opcode/literal features, package metadata, reference profile match, source validation, advisory match. |
| Provenance | Where did the evidence come from? | APK-side extraction, reference database, source repository validation, external vulnerability source, report-only model. |
| Coverage context | What references were available? | Present or missing coordinate versions, artifact-family coverage, extraction failures, source resolution status. |
| Transformation risk | What may have damaged the signal? | R8 shrinking, method inlining, package flattening, partial inclusion, multidex, hidden DEX, native payloads. |
| Quality profile | How reliable is the evidence? | Directness, uniqueness, independence, reproducibility, version specificity. |
| Uncertainty | What is unresolved? | Ambiguous adjacent versions, family-level match only, missing-source component, unsupported native package version. |
| Validation context | Where has the claim been checked? | Controlled fixture, clean real APK, partial real APK, source-known upper bound, diagnostic lane. |
| Permitted authority | How may it be used? | Confirmed SBOM component, potential component, report-only hint, confirmed vulnerability, potential vulnerability, out-of-scope boundary. |

### 4.3.2 Claim Granularity

The framework distinguishes four levels of component granularity:

1. **Identity claim:** a component family or coordinate is present, but the exact version is not established.
2. **Version claim:** a specific version or bounded version set is supported by version-discriminating evidence.
3. **Relationship claim:** the component is direct, transitive, bundled, copied, shaded, partially included, or uncertain.
4. **Security claim:** a vulnerability advisory applies, may apply, or does not apply under the current evidence.

This granularity is important because SBOM formats can represent package entries even when the recovery process is uncertain. The framework therefore separates syntactic representation from epistemic authority. A package URL can identify a candidate component, but the confidence class explains whether the PURL is exact, bounded, or only investigatory.

## 4.4 Evidence Hierarchy

The evidence hierarchy ranks evidence by what it can safely support. It is not a universal probability scale. Instead, it is a policy for deciding which interpretations are permitted when the available evidence is incomplete or conflicted.

**Table 4.2:** Evidence hierarchy for reconstructed component claims.

| Level | Meaning | Typical evidence | Permitted interpretation |
|---|---|---|---|
| E0 | No usable component evidence | No matching code, missing extraction, unsupported payload type, or unavailable reference family | No package-level claim. The result may record a boundary or extraction failure. |
| E1 | Weak contextual evidence | Package-name fragment, manifest hint, resource string, file name, or nearby ecosystem cue | Investigation hint only; not enough for an SBOM component claim. |
| E2 | Identity-supporting evidence | Class-set overlap, namespace pattern, partial structural match, or learned presence signal | Candidate component identity; exact version and vulnerability authority remain unavailable. |
| E3 | Independent multi-source identity evidence | Agreement between structural graph evidence, opcode/literal evidence, reference family metadata, or independent detector families | Strong identity claim, but exact version still requires version-discriminating evidence. |
| E4 | Version-discriminating APK evidence | Version-specific graph, opcode, literal, API, or stateful/stateless class features that separate nearby versions | Exact version or bounded version set, subject to conflict, coverage, and transformation caps. |
| E5 | Externally validated exact evidence | Source-resolved dependency tree, controlled fixture ground truth, reproducible build evidence, or manually verified exact-version evidence | Confirmed exact claim within the validation context; still not proof of global detector completeness. |

Two aspects of the hierarchy are deliberately strict. First, evidence levels are claim-specific. A signal may be E3 for identity but only E1 for exact version. Second, external

validation does not erase the conditions under which it was obtained. A controlled fixture can validate pipeline mechanics, while a source-validated real application can validate emitted versions for that application. Neither automatically proves that all real-world dependencies are recoverable.

## 4.5 Evidence Quality Dimensions

Evidence level describes what a signal may support. Evidence quality describes how fragile that support is. The framework evaluates each evidence item along the dimensions in table 4.3.

**Table 4.3:** Evidence quality dimensions used to bound claim confidence.

| Dimension | Trust question | Examples of risk |
|---|---|---|
| Directness | Does the signal observe the component itself or only context around it? | Manifest strings and package names are indirect; class and method structure are more direct. |
| Uniqueness | Does the signal distinguish one component or version from close neighbors? | Shared utility classes, shaded code, and adjacent AndroidX versions reduce uniqueness. |
| Independence | Do multiple signals fail together or independently? | Two features derived from the same package namespace are weaker than namespace plus graph plus literal evidence. |
| Version specificity | Can the evidence separate exact versions? | Identity features may confirm a coordinate while still leaving a version band. |
| Transformation resilience | Does the signal survive obfuscation and optimization? | R8 shrinking, inlining, class merging, and package flattening can remove or distort features. |
| Reference coverage | Were the relevant reference versions available and successfully extracted? | Missing historical versions or failed profiles cap confidence even when the best available candidate scores highly. |
| Provenance reliability | Is the evidence reproducible and auditable? | Manually curated notes, source-resolution logs, and generated feature artifacts carry different audit strength. |
| Conflict level | Do plausible alternatives remain? | Near-ties, artifact family ambiguity, and disagreement between source and APK-side evidence require explicit conflict handling. |

These dimensions allow a detector to be useful without being overconfident. For example, a learned Smali classifier may provide transformation-resilient identity evidence, but if it lacks exact-version specificity and independent validation, the framework keeps it report-only or identity-only. Conversely, source validation can strongly confirm emitted versions for one real application, but if the source tree contains dependencies that do not

appear in the compiled artifact, the result remains an emitted-version confirmation rate rather than full dependency recall.

## 4.6 Evidence Fusion, Conflict, and Abstention

### 4.6.1 Fusion Policy

The framework treats evidence fusion as an auditable decision process rather than a single opaque score. A practical implementation may still use numeric similarity scores, classifier outputs, and thresholds, but the final claim is assigned by policy:

$$\text{trust}(C) = f(S, D, I, V) - g(K, R, T)$$

This expression is conceptual. It should not be read as a calibrated probability unless a separate calibration study has been performed. Its role is to make the monotonicity rule explicit: more independent, direct, and version-specific evidence can increase trust, while unresolved conflict, missing references, and transformation risk can only cap or reduce it. Here, $S$, $D$, $I$, and $V$ denote support, directness, independence, and version specificity; $K$, $R$, and $T$ denote conflict, reference coverage risk, and transformation risk.

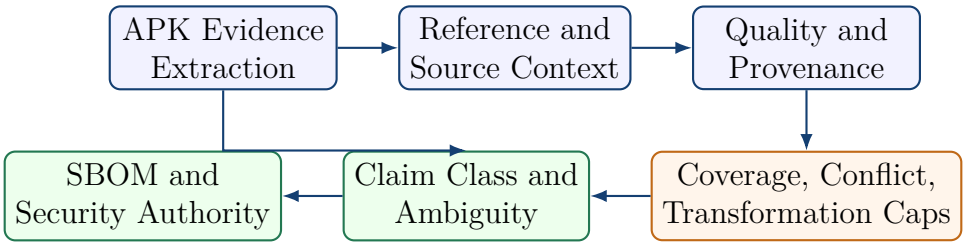



**Figure 4.1:** Conceptual flow of the proposed evidence-based trust framework.

### 4.6.2 Conflict Handling

Conflict is not an exceptional case in APK-side SBOM recovery; it is normal. Adjacent versions may share most bytecode, libraries may be partially bundled, old source trees may resolve different dependency versions from current reference repositories, and optimized builds may remove the features that would distinguish two candidates. The framework therefore defines the following conflict actions:

1. **Exact promotion:** promote a single exact version only when the selected candidate has version-discriminating evidence and plausible alternatives are ruled out by independent signals or validation.
2. **Bounded ambiguity:** emit a version set or band when the evidence supports a component family but cannot safely separate nearby versions.
3. **Identity-only output:** emit the component identity without a version when version evidence is weak, missing, or transformation-damaged.
4. **Report-only output:** preserve diagnostic candidates for human review or future experiments without changing authoritative SBOM fields.

5. **Abstention:** record no component claim when the evidence does not meet the minimum identity threshold or when the payload is outside the method's scope.

### 4.6.3 Confidence Caps

Confidence capping is the key mechanism that prevents overclaiming. A claim cannot exceed the weakest relevant boundary. For example, if the best available reference database lacks the source version, an exact-version match against a newer nearby reference may remain a candidate rather than a confirmed result. If a detector has only report-only validation, its result cannot modify authoritative CycloneDX components. If a native shared object is observed but no native package-version resolver exists, the framework may emit a file component with hash evidence but not a Maven or native package version claim.

## 4.7 Claim Classes

Table 4.4 defines the claim classes used by the framework. They are ordered by permitted authority, not by how useful they are. An identity-only or bounded claim can be more honest, and therefore more useful, than a false exact claim.

**Table 4.4:** Claim classes and permitted downstream use.

| Claim class | Required support | Permitted use |
|---|---|---|
| **Confirmed exact** | E5 validation or E4 evidence with strong independent corroboration and no unresolved nearby conflict | Authoritative SBOM component and exact-version-dependent security interpretation. |
| **Strong exact** | E4 version-discriminating evidence with adequate reference coverage and manageable transformation risk | Authoritative component entry with exact version, but validation context must be shown. |
| **Likely exact** | E4 evidence exists, but one quality dimension is weak or a limited alternative remains | Component entry may be emitted with confidence caveat; vulnerability findings should be conservative. |
| **Version-band ambiguous** | Identity is strong, but exact version cannot be separated from nearby candidates | Emit component with bounded version set or ambiguity field; exact CVE classification is not allowed without additional policy. |
| **Identity-only** | E2 or E3 identity evidence without version-discriminating support | Emit candidate component identity; do not emit exact-version CVE conclusions. |
| **Report-only candidate** | Diagnostic, experimental, or machine-learning evidence that has not passed authority gates | Human investigation, prioritization, or future work only. |
| **Unsupported or abstained** | Evidence below identity threshold or conflict too severe | No package claim; record reason for abstention if useful. |
| **Boundary or out-of-scope** | Payload is observed, but the method does not support its package or version semantics | File/hash component, limitation note, or risk flag rather than package-version recovery. |

## 4.8 Validation and Evidence Lanes

Evaluation results are meaningful only when their lane is explicit. The framework separates the lanes in table 4.5 so that strong evidence in one context is not accidentally generalized to another.

**Table 4.5:** Validation and evidence lanes.

| Lane | What it can show | What it must not imply |
|---|---|---|
| Controlled ground truth | Pipeline mechanics, exact expected versions, known transformations, and regression behavior under designed fixtures | Not a measure of real-world dependency completeness. |

| Lane | What it can show | What it must not imply |
|---|---|---|
| Clean source-validated real APKs | Exact emitted Maven versions can be checked against source-resolved dependency evidence for real applications | Not a global real-world score if the clean set is small or sparse. |
| Partial source-validated real APKs | Harder real applications can be analyzed with exact matches, mismatches, missing-source rows, and ambiguity separated | Partial evidence is not a total failure, but it is also not full recall. |
| Source-known upper bound | Source trees can indicate dependencies that may or may not be compiled into the APK | Source dependency presence alone does not prove APK-side presence. |
| Report-only diagnostic lane | Experimental signals can be inspected without changing authoritative results | Diagnostic improvement is not permission to alter SBOM or CVE outputs. |
| Experiment-only optimization lane | Narrow fixes can be tested with controls, near-tie analysis, and regression checks | A scoped preflight is not a shared default until controls and evidence gates pass. |
| Limitation or boundary lane | Native payloads, hidden code, unsupported transformations, and extraction failures can be reported honestly | Boundary cases must not be counted as Maven-version failures or successes. |

This lane structure also changes how metrics should be reported. A controlled fixture success rate, a clean real-APK emitted-version confirmation rate, and a partial real-APK confirmation rate answer different questions. Combining them into one number can be useful only after the meaning and denominator are made explicit. In particular, an emitted Maven version confirmation rate should not be described as full dependency recall unless the evaluation has shown that all relevant source dependencies are present in the compiled artifact and within scope.

## 4.9 Vulnerability-Claim Policy

CVE and advisory mapping is downstream of component recovery. The framework therefore applies an authority rule: vulnerability status cannot be stronger than the recovered component and version evidence. This is especially important when advisory ranges are narrow, when adjacent versions have different security status, or when the result is identity-only.

**Table 4.6:** Vulnerability interpretation policy.

| Component evidence | Allowed vulnerability status | Rationale |
|---|---|---|
| Confirmed exact vulnerable version | Confirmed affected | The version claim has sufficient authority to evaluate version-specific advisory ranges. |
| Confirmed exact non-affected version | Confirmed not affected for that advisory | The result is advisory-specific; absence of one advisory does not prove the component is risk-free. |
| Likely exact version | Potential affected or likely affected, depending on policy | The version may be used for triage, but the confidence caveat remains visible. |
| Version-band ambiguous | Potential affected if any candidate in the band is affected | Exact affected/not-affected labels are unsafe unless all candidates share the same advisory status. |
| Identity-only component | Potential exposure only when advisory identity matches and version is unknown | Version-specific claims are not permitted. |
| Report-only candidate | Report-only vulnerability hint | The finding may guide investigation but must not change authoritative security status. |
| Boundary or out-of-scope payload | File/hash risk note or external analysis required | Package-version advisory mapping is unavailable in the current method. |

This policy supports conservative security reporting. It avoids two common errors: hiding a possible issue because the exact version is unknown, and declaring a confirmed vulnerability from evidence that only supports component identity. The result can still be actionable: a potential finding may trigger manual review, dependency upgrade investigation, or source validation, while a confirmed finding can support stronger remediation language.

## 4.10 Recovered Claim Lifecycle

The framework can be implemented as a claim lifecycle with explicit state transitions:

1. **Extract evidence** from the APK, including bytecode, resources, manifests, native files, and available build metadata.
2. **Generate candidates** from reference profiles, structural features, learned hints, package metadata, or source-known context.
3. **Normalize identifiers** into coordinates, package URLs, file hashes, advisory identifiers, and relationship candidates.
4. **Attach provenance** for every evidence item and candidate source.

5. **Assess quality and coverage** by checking directness, independence, version specificity, reference availability, and transformation exposure.
6. **Fuse evidence** while preserving conflicts, near-ties, and missing-reference explanations.
7. **Assign claim class** with confidence, ambiguity, validation lane, and permitted downstream authority.
8. **Emit or abstain** in the SBOM, vulnerability report, and diagnostic artifacts according to the assigned authority.

The lifecycle is intentionally auditable. A reviewer should be able to trace a CycloneDX component entry or vulnerability row back to the features, reference profiles, source validation, and policy decisions that produced it.

## 4.11 PFE System as a Case Study

The PFE implementation associated with this thesis provides a concrete case study for the framework. The goal of this section is not to present final evaluation results; those belong in the project evaluation artifacts and the general conclusion. Instead, the case study shows how the proposed trust model organizes evidence from a real APK-to-SBOM pipeline.

**Table 4.7:** Mapping PFE observations to framework concepts.

| Case-study observation | Framework concept | Safe interpretation |
|---|---|---|
| Controlled transformed fixtures with known expected Maven versions | Controlled ground-truth lane and E5 validation | Demonstrates pipeline mechanics and transformation-specific regressions under designed conditions; does not prove complete real-world recovery. |
| Clean source-validated real APKs with exact emitted Maven-version checks | Clean real-APK validation lane | Confirms emitted versions for those applications and denominators; should remain separate from partial and sparse cases. |
| Partial source-validated real APKs with exact matches, version mismatches, and missing-source rows separated | Partial real-APK lane, conflict preservation, and emitted-version confirmation rate | Shows useful recovery in difficult applications without presenting unresolved rows as either success or total failure. |

| **Case-study observation** | **Framework concept** | **Safe interpretation** |
|---|---|---|
| Reference coverage and matcher recall measured as separate gaps | Reference availability, candidate-generation coverage, and denominator separation | Shows whether failure arises because the true release is absent from the reference base or because available evidence was not selected; neither gap should be hidden inside one accuracy value. |
| Source-known constrained reruns with exact source versions available | Source-known upper-bound lane | Demonstrates recognition under a favorable candidate space; does not measure unconstrained APK-only accuracy or prove that every source dependency survives in the binary. |
| Version-band ambiguity and adjacent-version near-ties | Bounded ambiguity, confidence caps, and experiment-only optimization lane | Prevents a weak near-tie from becoming a confirmed exact version or a confirmed CVE status. |
| Broad scoring and reference-database changes that improved targets but regressed controls | Negative evidence and promotion governance | Preserves failed experiments as evidence that a signal is unsafe for shared authority, even when it is useful diagnostically. |
| The scoped AndroidX Fragment 1.6.2 preflight selected the intended release without sampled control regressions, but lacked version-unique APK evidence | Experiment-only near-tie decision | Keeps the result outside the shared reference database because clean sampled controls do not convert feature-equivalent candidates into exact evidence. |
| Report-only R8 and AI assistance | Report-only containment | Allows learned or diagnostic signals to support triage while keeping authoritative SBOM and vulnerability outputs deterministic until validation gates pass. |
| Native shared objects and hidden-code cases | Boundary or out-of-scope claim class | Supports file/hash evidence, limitation notes, or risk flags without inventing Maven or native package-version claims. |

This mapping illustrates the main contribution of the framework: it turns mixed evidence into bounded statements. A controlled result can be strong without being overgeneralized. A partial real-APK result can be valuable without being hidden as a failure. A source-known dependency can inform an upper bound without proving APK-side presence. A report-only classifier can guide investigation without becoming SBOM authority. These distinctions make the pipeline more cautious, but they also make it more usable because users can decide how to act on each claim class.

## 4.12 Framework Evaluation

The framework should be evaluated at two levels. The first level asks whether the framework improves the interpretability of recovered SBOMs. The second asks whether it prevents unsafe promotion of weak evidence. Suitable evaluation questions include:

1. Can a reviewer trace every emitted component and vulnerability finding back to evidence, provenance, and validation context?
2. Are identity-only, version-band, report-only, and boundary outcomes preserved in the output rather than collapsed into exact versions?
3. Do controlled, clean real, partial real, diagnostic, and limitation lanes remain separated in metrics and narrative claims?
4. When evidence conflicts, does the framework abstain or bound the result instead of selecting an unsupported exact version?
5. Do vulnerability labels obey the authority of the underlying component and version claims?
6. Can future detector improvements be added without changing the trust policy or silently increasing downstream authority?

These questions complement traditional precision, recall, and F1 metrics. A detector may score well under a narrow benchmark while still producing an untrustworthy SBOM if it hides ambiguity or provenance. Conversely, a conservative system may have a lower exact-version emission rate but provide more reliable security decisions because it exposes uncertainty and abstains when necessary.

## 4.13 Limitations and Research Agenda

The proposed framework is intentionally evidence-centric, but it has several limitations.

1. **It does not solve feature extraction by itself.** The framework requires detectors, reference profiles, source validation, and vulnerability data. Weak upstream evidence still limits the final claims.
2. **It does not provide calibrated probabilities.** Confidence classes are policy categories unless supported by a separate calibration study over representative datasets.
3. **It depends on reference and source availability.** Missing Maven versions, unavailable historical artifacts, broken build systems, and incomplete source trees can cap evidence even when the APK-side matcher is strong.
4. **It does not fully solve transitive dependency semantics.** Reconstructed bytecode can show that code is present, but distinguishing direct, transitive, shaded, copied, and build-time-only dependencies remains difficult.

5. **It leaves native package-version recovery as future work.** Native files can be represented as file components with hashes, but mapping them to precise upstream package versions requires a different evidence base.

6. **It requires disciplined reporting.** The framework is only useful if evaluation tables, thesis claims, and generated reports keep the lanes and authority boundaries visible.

These limitations define a research agenda. Future work should combine multi-scale bytecode evidence with calibrated confidence models, richer reference coverage, reproducible source-validation pipelines, native library package recovery, and benchmark protocols that measure not only exact matches but also abstention quality and vulnerability-authority correctness.

## 4.14 Conclusion

This chapter proposed an evidence-based trust framework for reconstructing SBOM claims from compiled Android applications. The framework treats each component, version, relationship, and vulnerability statement as an auditable claim with evidence, provenance, quality dimensions, validation context, ambiguity, and permitted downstream authority. Its main purpose is to prevent weak or partial evidence from being consumed as stronger security knowledge.

The framework also provides a bridge between the literature reviewed in the previous chapter and the PFE APK-to-SBOM implementation. Existing detection techniques supply useful signals; the framework defines how those signals are bounded, fused, and reported. This makes trustworthy SBOM recovery less a question of always producing one exact answer and more a question of producing the strongest answer that the evidence can honestly support.

# Part V

# General Conclusion

# Chapter 5

# General Conclusion and Future Directions

## 5.1 General Conclusion

Recovering an SBOM from a compiled Android application is not only a detection problem. It is a problem of deciding what can be claimed from incomplete, transformed, and sometimes conflicting evidence. A component name or exact version may look precise while depending on weak namespace overlap, an incomplete reference collection, a feature-equivalent adjacent release, or source information that does not prove binary presence. Trust therefore depends on how evidence is qualified and bounded, not merely on whether a tool returns an answer.

Chapter 2 established the distinction between build-generated and reconstructed SBOMs and defined the methodological conditions for reviewing binary component-recovery research. The taxonomy in Chapter 3 then organized the literature by recovery role, evidence source, transformation exposure, version capability, validation practice, and trust implication. This synthesis showed that the field offers many useful technical signals, but does not provide one evidence model that connects those signals to provenance, ambiguity, validation context, and permitted security interpretation.

Chapter 4 addressed that gap with an evidence-based trust framework. The framework treats every recovered component, version, relationship, and vulnerability statement as a structured claim. Each claim records its evidence, provenance, coverage, transformation exposure, confidence class, unresolved alternatives, validation lane, and downstream authority. Evidence may be fused when it is sufficiently independent and claim-relevant, but authority is capped by missing references, unresolved conflicts, weak version specificity, and unsupported payload types. Abstention is therefore a valid result rather than an implementation failure.

The associated PFE system was used as a bounded case study, not as a second set of master experiments. Its controlled, clean real, partial real, source-known, experiment-only, report-only, and boundary observations demonstrate why a single success rate cannot describe recovered-SBOM trust. They also show the practical value of preserving failed promotions, near-ties, native files, and hidden-code risks as distinct evidence outcomes. The case study supports the framework's representational coverage while leaving broader independent validation as future work.

The central answer of this thesis is consequently conditional: heterogeneous APK evidence can support trustworthy reconstructed SBOM claims when the system preserves provenance, evaluates evidence at the identity and version levels separately, exposes conflict and missing coverage, distinguishes validation lanes, and prevents downstream vulnerability authority from exceeding the underlying component claim. Trustworthiness is not synonymous with always selecting an exact version. It is the ability to produce the strongest claim that the available evidence can justify, while making the remaining uncertainty visible and actionable.

## 5.2 Answers to the Research Questions

Table 5.1 summarizes the answers to the six supporting research questions introduced in Chapter 1.

**Table 5.1:** Answers to the supporting research questions.

| RQ | Question focus | Answer |
|---|---|---|
| RQ1 | Evidence sources available in compiled Android applications | Recoverable evidence spans archive and manifest metadata, DEX package and class structure, methods, opcodes, literals, API references, dependency and call relations, resources, native-file observations, versioned reference artifacts, publication metadata, and external validation such as reproducible fixtures or source-resolved dependency trees. These sources differ in directness, version specificity, resilience, and authority. |
| RQ2 | How existing approaches combine evidence | Existing approaches range from namespace and class-set matching to structural graphs, opcode and semantic features, fuzzy signatures, learned representations, metadata coherence, and multi-stage reference matching. The literature supports multi-source reasoning, but no combination is universally reliable under shrinking, optimization, repackaging, feature-equivalent releases, and incomplete references. Fusion must therefore remain explicit, scoped, and subject to controls. |
| RQ3 | Factors that determine identity and version strength | Evidence strength depends on directness, uniqueness, independence, transformation resilience, version specificity, provenance reliability, reference coverage, and reproducibility. Identity and exact-version support must be assessed separately because a signal can identify a library family while remaining unable to distinguish adjacent releases. |

| RQ | Question focus | Answer |
|---|---|---|
| RQ4 | Representation of conflict, absence, transformation, and source dependence | The framework represents these conditions through competing candidates, bounded version sets, conflict records, coverage and transformation caps, explicit validation lanes, report-only outcomes, boundary classes, and abstention. Missing evidence is not interpreted as component absence, and source-known dependencies are not treated as proof of APK-side presence. |
| RQ5 | Information required for an auditable and security-relevant claim | A claim must identify the subject APK, candidate component and artifact family, claim type and granularity, version or version set, supporting and opposing evidence, provenance, coverage, transformation exposure, confidence class, ambiguity, validation context, and permitted downstream use. Vulnerability interpretation must inherit these boundaries rather than bypass them. |
| RQ6 | Evaluation without overstating accuracy | Evaluation should separate controlled, clean source-validated, partial source-validated, source-known upper-bound, diagnostic, experiment-only, and limitation lanes. It should publish the relevant denominator, distinguish emitted-row correctness from full dependency recall, retain negative experiments and abstentions, test traceability and authority preservation, and require target/control evidence before promoting a signal. |

Together, these answers shift evaluation from the question "Did the detector emit a version?" to the stronger question "What evidence supports this claim, what could invalidate it, and how may it safely be used?"

## 5.3 Main Contributions

The thesis makes six connected contributions.

1. **A recovery-oriented taxonomy.** The reviewed work is organized by recovery objective, evidence family, transformation threat, version capability, evaluation practice, and resulting trust implication rather than by headline performance alone.
2. **Operational trust requirements.** The thesis defines trustworthy reconstruction through provenance, evidence visibility, identity/version separation, uncertainty preservation, validation context, bounded vulnerability authority, and explicit denominators.
3. **A structured recovered-claim model.** Component identities, exact or bounded versions, relationships, native observations, and vulnerability statements are represented as auditable claims with evidence and permitted authority.

4. **An evidence hierarchy and fusion policy.** Evidence is ranked by what it may support, evaluated across quality dimensions, fused only under independence and relevance constraints, and capped when conflicts, transformation risk, or missing coverage remain.
5. **A validation and vulnerability-authority model.** The framework separates evidence lanes and ensures that a vulnerability statement cannot be stronger than the component identity and version evidence from which it is derived.
6. **A PFE-informed framework case study.** Selected project observations demonstrate that the model can represent exact success, partial recovery, source-known upper bounds, near-ties, failed promotions, report-only assistance, and unsupported native or hidden-code regions without presenting them as equivalent evidence.

The conceptual contribution is independent of the PFE architecture. A future detector could replace the current feature extractor, matcher, or learned assistant while retaining the same claim schema, evidence-quality questions, validation lanes, and authority rules.

## 5.4 Limitations

### 5.4.1 Review Scope and Evidence Availability

The structured review is concentrated on Android and closely related binary component-recovery research. The Tier A corpus is grounded in locally available full text, while Tier B records remain useful discovery evidence but require additional verification before supporting strong comparative claims. The review should therefore be understood as structured and reproducible within its recorded scope, not as proof that every relevant publication has been captured.

### 5.4.2 Domain and Ecosystem Scope

The framework is derived mainly from DEX, Maven, Android build behavior, and the evidence problems encountered in compiled APKs. Its general principles may transfer to other ecosystems, but evidence hierarchies and validation rules would need adaptation for native packages, JavaScript bundles, containers, firmware, or platform-specific package managers.

### 5.4.3 Uncalibrated Confidence Classes

The proposed confidence and evidence levels are policy categories, not universally calibrated probabilities. They define permitted interpretations and useful abstention behavior, but do not state that a class has the same empirical error rate across component families, application eras, or transformation profiles.

### 5.4.4 Case-Study Dependence

The PFE provides varied and traceable scenarios, yet it was designed by the same project context that informed the framework. This creates a risk that the claim classes fit the

available artifacts especially well. Independent tools, datasets, and analyst studies are needed to test whether the framework remains complete, understandable, and useful outside this implementation.

### 5.4.5 Source and Reference Boundaries

External source validation is strong evidence for declared or resolved dependencies, but it is not a perfect manifest of code present in the APK. Shrinking, optional dependencies, metadata-only artifacts, vendoring, and build-time tooling complicate the relation between source trees and binary content. Likewise, a reference-based detector cannot identify an exact release that is absent from its reference base.

### 5.4.6 Native, Dynamic, and Protected Code

The framework can represent native files, hidden payload indicators, and unsupported transformations as boundary evidence. It does not solve native package/version attribution, runtime-loaded dependency inventory, encrypted payload recovery, or commercial protector analysis. These areas require different evidence sources and validation protocols.

## 5.5 Future Research

**Table 5.2:** Priority directions for extending the trust framework.

| Direction | Research objective | Required evidence |
|---|---|---|
| Empirical calibration | Calibrate identity, exact-version, ambiguity, and abstention classes by component family and transformation profile | Held-out ground truth, reliability analysis, and error-sensitive calibration metrics. |
| Independent framework validation | Apply the claim model and authority policy to other detectors, analysts, and datasets | External tools, blinded case review, and inter-rater agreement on claim classes. |
| Public benchmark design | Measure exactness, coverage, abstention, provenance, and vulnerability-authority correctness together | Reproducible APKs, complete denominators, transformation labels, and version-discriminating ground truth. |
| Provenance-aware SBOM profiles | Standardize how reconstructed claims expose evidence, ambiguity, and validation context | Interoperable CycloneDX or related profiles tested across producers and consumers. |
| Evidence knowledge graphs | Connect coordinates, aliases, target variants, releases, binary evidence, source observations, and vulnerability ranges | Provenance-preserving graph schemas with controls against metadata-only overconfidence. |

| Direction | Research objective | Required evidence |
|---|---|---|
| Native and dynamic attribution | Extend claims to shared libraries, bundled runtimes, protected payloads, and downloaded code | Separate corpora, architecture-aware fingerprints, runtime provenance, and scope-specific metrics. |
| Human factors and explainability | Determine whether analysts understand and act appropriately on claim classes, conflicts, and abstentions | User studies, decision-quality measures, and explanation comparisons. |

### 5.5.1 Calibrated and Set-Valued Claims

A first priority is to replace purely ordinal confidence classes with empirically evaluated calibration where representative labels are available. Calibration should remain claim-specific: identity probability, exact-version probability, and affected-range confidence answer different questions. Set-valued version predictions are also promising for releases that are indistinguishable under surviving APK evidence. Their quality should be measured by coverage and set size rather than by rewarding arbitrary single-version guesses.

### 5.5.2 Independent and Human-Centered Evaluation

The framework should be tested with analysts who did not design it and with detectors that expose different evidence families. Such studies can measure whether reviewers assign consistent claim classes, trace outputs to the same provenance, recognize when vulnerability authority is capped, and make safer decisions than users of a binary present/absent report.

### 5.5.3 Interoperable Provenance

Reconstructed SBOM producers need a shared vocabulary for observed binary evidence, inferred identity, candidate version sets, external validation, and abstention. Future work can explore CycloneDX extensions or profiles that preserve these distinctions while remaining consumable by vulnerability management and policy tools. Cross-tool evidence exchange would also allow an analyst to combine independent signals without erasing their origin.

### 5.5.4 Broader Component Domains

Native, dynamic, and protected components should be treated as separate research domains rather than forced into Maven-oriented claims. Native attribution may use symbols, strings, sections, function fingerprints, build identifiers, and architecture-normalized features. Runtime collection should record where a payload was obtained, when it was loaded, and whether it was packaged, unpacked, generated, or downloaded.

### 5.5.5 Evidence Knowledge Graphs

A provenance-aware knowledge graph could connect package coordinates, target variants, release families, reference artifacts, binary observations, source validation, experiments, and vulnerability ranges. The graph could explain why a candidate was considered and which evidence conflicts with it. Its relations must remain supporting evidence rather than a substitute for observed APK content.

## 5.6 Closing Remarks

A recovered SBOM is trustworthy only when its precision of language matches the precision of its evidence. Exact claims should be exact for a reason; ambiguous claims should remain ambiguous; unsupported regions should be visible; and vulnerability conclusions should stop where component evidence stops. The long-term objective is therefore not a system that always returns a complete dependency list. It is a system that can explain why each returned claim should be believed, what remains unknown, and how the result may safely guide action.

# Appendix A

# Literature Review Protocol

This appendix records version 1.0 of the structured literature mapping protocol, frozen on 21 June 2026. It is the operational companion to the methodology summarized in chapter 2. The review follows the planning, selection, quality-assessment, extraction, and synthesis principles of software-engineering review guidance [5], while explicitly acknowledging that the seed corpus was assembled iteratively before the protocol was frozen.

## A.1 Research Questions

**RQ-R1** Which evidence sources are used to identify Android third-party components from compiled applications?

**RQ-R2** Which techniques support library identity, exact-version attribution, or both?

**RQ-R3** Which compilation, optimization, obfuscation, packaging, and dependency transformations threaten each evidence source?

**RQ-R4** How do studies construct ground truth and evaluate library-level, version-level, and vulnerability-related claims?

**RQ-R5** How do studies represent confidence, ambiguity, provenance, abstention, partial inclusion, and conflicting evidence?

**RQ-R6** Which research gaps prevent recovered component claims from becoming trustworthy and auditable SBOM statements?

## A.2 Search Strategy

### A.2.1 Sources and Scope

The primary search sources are ACM Digital Library, IEEE Xplore, SpringerLink, ScienceDirect, USENIX, and arXiv. Google Scholar is used for discovery and backward or forward citation chaining rather than as the sole bibliographic authority. The publication window begins in 2015 and ends on the date recorded for each search refresh. English full text is preferred; English title and abstract are mandatory for screening.

### A.2.2 Concept Blocks

**Table A.1:** Search concept blocks

| Block | Terms |
|---|---|
| Platform | Android, APK, DEX, Dalvik |
| Component | third-party library, TPL, software component, dependency, SBOM, software bill of materials |
| Task | detect*, identif*, recogn*, recover*, attribut*, fingerprint*, match* |
| Threat or specificity | obfuscat*, R8, ProGuard, optim*, shrink*, inlin*, version*, vulnerab* |

The canonical query is:

```
(Android OR APK OR DEX OR Dalvik)
AND ("third-party library" OR TPL OR "software component" OR dependency
   OR SBOM OR "software bill of materials")
AND (detect* OR identif* OR recogn* OR recover* OR attribut*
   OR fingerprint* OR match*)
AND (obfuscat* OR R8 OR ProGuard OR optim* OR shrink* OR inlin*
   OR version* OR vulnerab*)
```

Database-specific syntax may change, but a refresh must preserve the four concept blocks and record the exact executed query, date, raw result count, deduplicated count, and selection decisions.

## A.3 Inclusion and Exclusion Criteria

### A.3.1 Inclusion

A publication is included when it:

1. addresses component, library, package, or version recovery from Android binaries or evaluates tools that do;
2. studies evidence obtainable from compiled artifacts, reference libraries, package metadata, or related dependency knowledge;
3. contributes a method, benchmark, empirical assessment, taxonomy, or SBOM-oriented framework relevant to compiled-artifact attribution;
4. permits extraction of at least the task, evidence source, evaluation basis, and limitations; and
5. has an accessible English title and abstract.

### A.3.2 Exclusion

A publication is excluded when it:

1. relies only on source dependency manifests and does not study binary recovery;
2. concerns malware classification, clone detection, or vulnerability scanning without a component-attribution contribution;
3. is a tutorial, slide deck, blog post, product page, or duplicate;
4. reports no separable Android result;
5. lacks enough information to identify its method or evaluation basis; or
6. is a shorter version of an included extended study and contributes no distinct method, dataset, or result.

### A.3.3 Screening and Evidence Tiers

Screening proceeds through deduplication, title-and-abstract screening, full-text screening, and tier assignment. The evidence tiers are:

**Tier A — full-text verified** A local full text has been inspected and the publication may support structured and quantitative extraction.

**Tier B — supplementary record** Metadata or a structured note exists, but the local full text has not yet been verified. The record may guide search and provisional gap mapping, but not precise quantitative claims.

**Tier C — contextual authority** Standards, official documentation, and project artifacts used for definitions or framework design. They are not counted as primary studies.

**Excluded** The reason for exclusion is recorded and the item is not used in synthesis.

## A.4 Data Extraction Schema

**Table A.2:** Structured extraction fields

| Dimension | Extracted information |
|---|---|
| Bibliographic identity | Title, authors, year, venue, DOI or stable URL, publication type, study family, evidence tier. |
| Objective and task | Research objective; family, library, class, version, SBOM, or vulnerability granularity. |
| Inputs and assumptions | Required APK artifacts, reference corpus, metadata, training data, and coverage assumptions. |
| Segmentation | Package boundaries, class hierarchy, dependency graph, clustering, or other module-decoupling method. |

| Dimension | Extracted information |
|---|---|
| Evidence | Names, APIs, opcodes, control flow, call/dependency graphs, literals, resources, native code, metadata, learned representations, and cross-component relations. |
| Decision method | Hashing, similarity, graph matching, ranking, classification, probability, learned model, or evidence fusion. |
| Uncertainty behavior | Threshold, confidence, competing versions, abstention, partial matching, provenance, and conflict handling. |
| Transformations | Renaming, package flattening, shrinking, optimization, control-flow rewriting, method inlining, repackaging, multidex, partial inclusion, native code, and dynamic loading. |
| Evaluation | Dataset, ground truth, evaluation unit, metrics, baselines, obfuscation setting, and independent benchmark results. |
| Operational support | Artifact availability, runtime, scalability, SBOM format, vulnerability mapping, and transitive-dependency handling. |
| Limitations and relevance | Threats to validity, negative findings, and adoption, adaptation, or rejection relevance to the framework. |

Unreported information is recorded as `NR`; it is not inferred from neighboring fields or from another publication in the study family.

## A.5 Quality Assessment

Tier A publications receive a score of 1, 0.5, or 0 for each question:

1. Is the research objective clear?
2. Is the method described well enough to understand or reproduce?
3. Are the dataset and ground truth described?
4. Are the task unit and metrics explicit?
5. Are transformation or obfuscation conditions stated?
6. Are identity and exact-version claims separated?
7. Is an implementation, dataset, or artifact available?
8. Are limitations or threats to validity acknowledged?

The score controls synthesis weight rather than automatic inclusion. Historically important studies may remain in the map with an explicit reporting-quality limitation.

## A.6 Synthesis Rules

- Narrative and taxonomy-based synthesis is the default.
- Incompatible datasets and metrics are not pooled into a common leaderboard.

- Library-level, class-level, and version-level results remain separate.
- Original-paper results remain separate from independent benchmark results.
- Obfuscation and optimization conditions must accompany performance claims.
- Artifact availability is assessed independently of reported accuracy.
- Tier B records cannot independently support precise numerical claims.
- Identity authority, version authority, and vulnerability authority are separate.
- Explicit ambiguity or abstention is preferred to unsupported exactness.

## A.7 Selected Studies

**Table A.3:** Frozen seed corpus and current evidence status

| ID | Publication or study label | Year | Status |
|---|---|---|---|
| S01 | LibScout [8] | 2016 | Tier A |
| S02 | LibD [11] | 2017 | Tier A |
| S03 | LibPecker [12] | 2018 | Tier A |
| S04 | ORLIS [9] | 2018 | Tier A |
| S05 | LibID [10] | 2019 | Tier A |
| S06 | Automated Third-Party Library Detection: Are We There Yet? [6] | 2020 | Tier A |
| S07 | Systematic Assessment of Android TPL Detection Tools [7] | 2021 | Tier A |
| S08 | ATVHUNTER [13] | 2021 | Tier A |
| S09 | LibScan [14] | 2023 | Tier A |
| S10 | Empirical Study of TPL Detection in iOS and Android [18] | 2024 | Tier A |
| S11 | LibHunter / Code-Optimization Impact Study [15] | 2024 | Tier A |
| S12 | LibAttention [17] | 2025 | Tier A |
| S13 | SAD [16] | 2025 | Tier A |
| S14 | LibRadar | 2016 | Tier B |
| S15 | Libra | 2023 | Tier B |
| S16 | LibMD | 2025 | Tier B |
| S17 | TPL-Benchmark / Revisiting TPL Detection | 2025 | Tier B |
| S18 | UniBOM | 2025 | Tier B |
| S19 | LibScope / Towards Global Matches | 2026 | Tier B |
| S20 | ObfDetector / Advanced-Obfuscation Robustness Study | 2026 | Tier B |

The seed corpus contains 20 publication units: 13 Tier A full texts and seven Tier B supplementary records. This count is a protocol checkpoint, not a claim that all 20 have equal evidential authority. A Tier B record is promoted only after full-text verification, bibliographic checking, structured extraction, and quality assessment.

## A.8 Audit and Change Control

The current extraction records are maintained under `papers/data/`. Each search refresh must append the date, sources, query variants, raw and deduplicated result counts, inclusion and exclusion decisions, tier changes, and taxonomy effects to the master-thesis session log. Eligibility, evidence tiers, and synthesis rules may be amended only through a dated protocol change note; silent changes are prohibited.

# Bibliography


[1] SPDX Workgroup. *SPDX Specifications*. URL: https://spdx.dev/use/specifications/ (visited on 06/19/2026).

[2] OWASP CycloneDX Project. *CycloneDX Specification*. URL: https://cyclonedx.org/specification/overview/ (visited on 06/19/2026).

[3] Package URL Project. *Package URL Specification*. URL: https://github.com/package-url/purl-spec (visited on 06/19/2026).

[4] Android Developers. *Shrink, Obfuscate, and Optimize Your App*. URL: https://developer.android.com/build/shrink-code (visited on 06/19/2026).

[5] Barbara Kitchenham and Stuart Charters. *Guidelines for Performing Systematic Literature Reviews in Software Engineering*. Tech. rep. EBSE-2007-01. Keele University and Durham University, 2007. URL: https://www.elsevier.com/__data/promis_misc/525444systematicreviewsguide.pdf (visited on 06/21/2026).

[6] Xian Zhan et al. "Automated Third-Party Library Detection for Android Applications: Are We There Yet?" In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 2020, pp. 919–930. DOI: 10.1145/3324884.3416582.

[7] Xian Zhan et al. "A Systematic Assessment on Android Third-Party Library Detection Tools." In: *arXiv preprint arXiv:2108.01964* (2021). DOI: 10.48550/arXiv.2108.01964.

[8] Michael Backes, Sven Bugiel, and Erik Derr. "Reliable Third-Party Library Detection in Android and Its Security Applications." In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 2016, pp. 356–367. DOI: 10.1145/2976749.2978333.

[9] Yan Wang et al. "ORLIS: Obfuscation-Resilient Library Detection for Android." In: *Proceedings of the 5th IEEE/ACM International Conference on Mobile Software Engineering and Systems*. 2018. DOI: 10.1145/3197231.3197248.

[10] Jiexin Zhang, Alastair R. Beresford, and Stephan A. Kollmann. "LibID: Reliable Identification of Obfuscated Third-Party Android Libraries." In: *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 2019. DOI: 10.1145/3293882.3330563.

[11] Menghao Li et al. "LibD: Scalable and Precise Third-Party Library Detection in Android Markets." In: *Proceedings of the 39th International Conference on Software Engineering*. 2017, pp. 335–346. DOI: 10.1109/ICSE.2017.38.

[12] Yuan Zhang et al. "Detecting Third-Party Libraries in Android Applications with High Precision and Recall." In: *Proceedings of the 25th IEEE International Conference on Software Analysis, Evolution and Reengineering*. 2018, pp. 141–152. DOI: 10.1109/SANER.2018.8330204.

[13] Xian Zhan et al. "ATVHUNTER: Reliable Version Detection of Third-Party Libraries for Vulnerability Identification in Android Applications." In: *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering*. 2021, pp. 1695–1707. DOI: 10.1109/ICSE43902.2021.00150.

[14] Yafei Wu et al. "LibScan: Towards More Precise Third-Party Library Identification for Android Applications." In: *32nd USENIX Security Symposium*. 2023. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/wu-yafei (visited on 06/20/2026).

[15] Zifan Xie et al. "How Does Code Optimization Impact Third-Party Library Detection for Android Applications?" In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 2024. DOI: 10.1145/3691620.3695554.

[16] Bolin Zhou et al. "Version-Level Third-Party Library Detection in Android Applications via Class Structural Similarity." In: *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*. 2025, pp. 604–615. DOI: 10.1145/3756681.3756933.

[17] Dahan Pan et al. "Pay Your Attention on Lib! Android Third-Party Library Detection via Feature Language Model." In: *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering*. 2025, pp. 217–228. DOI: 10.1109/SANER64311.2025.00028.

[18] Pablo Marchand Ramirez. "Empirical Study of Third-Party Library Detection in iOS and Android Applications." MA thesis. Universidad Politécnica de Madrid, 2024.